

第一章 绪论

一、基本概念:

数据是数据库存储的基本对象,它包括数字、文字、声音、图象、动画等。数据(Data)是数据库中存储的基本对象

数据库: Database(DB)是长期存储在计算机内、有组织的、可共享的数据集合。

数据库的基本特征 (基本特点: 长久存储, 有组织, 可共享)

数据按一定的数据模型组织、描述和储存; 可为各种用户共享; 冗余度较小; 数据独立性较高; 易扩展

数据库管理系统: Database Management System 又称 DBMS, 是位于用户与操作系统之间的一层数据管理软件。

DBMS 的主要功能:

- 数据定义功能: 数据组织、存储和管理;
- 数据库的事务管理和运行管理;
- 数据库的建立和维护功能(实用程序);
- 其它功能: DBMS 与网络中其它软件系统的通信, 两个 DBMS 系统的数据转换, 异构数据库之间的互访和互操作

数据库系统: Database System (DBS) 在计算机系统中引入数据库后的系统构成

数据库系统的构成:

数据库 DB, 数据库管理系统 (及其开发工具) DBMS, 应用系统, 数据库管理员 DBA

数据管理技术的产生和发展: 人工管理阶段, 文件系统阶段, 数据库系统阶段

数据库系统阶段特点:

- **数据结构化:** 数据的最小存取单位是数据项, 易于扩展数据内容
- **数据的共享性高,** 冗余度低, 易扩充(数据共享的好处: 减少数据冗余, 节约存储空间, 避免数据之间的不相容性与不一致性, 使系统易于扩充。)
- **数据独立性高:** 物理独立性: 指用户的应用程序与存储在磁盘上的数据库中数据是相互独立的。物理存储改变不影响应用程序。逻辑独立性: 指用户的应用程序与数据库的逻辑结构是相互独立的。存储逻辑改变, 不影响应用。反之亦然。
- **数据由 DBMS 统一管理和控制。** 数据的安全性 (Security) 保护(防止数据的非法使用), 数据的完整性 (Integrity) 检查 (数据的正确、有效和相容), 并发 (Concurrency) 控制 (多用户并发访问的控制和协调), 数据库恢复 (Recovery) (由于意外故障的数据恢复。保证数据库的正确状态)。

数据库系统的结构分为 (管理系统角度): 外模式、模式、内模式。

模式 (也称逻辑模式): 数据库中全体数据的逻辑结构和特征的描述, 所有用户的公共数据视图, 综合了所有用户的需求, 一个数据库只有一个模式。是数据库系统模式结构的中间层, 与数据的物理存储细节和硬件环境无关, 与具体的应用程序、开发工具及高级程序设计语言无关

外模式 (也称子模式或用户模式) 数据库用户使用的局部数据的逻辑结构和特征的描述, 数据库用户的数据视图, 是与某一应用有关的数据的逻辑表示。 外模式的地位: 介于模式与应用之间

模式与外模式的关系: 一对多。外模式通常是模式的子集, 一个数据库可以有多个外模式。反映了不同的用户的应用需求、看待数据的方式、对数据保密的要求, 对模式中同一数据, 在外模式中的结构、类型、长度、保密级别等都可以不同。

内模式 (也称存储模式): 是数据物理结构和存储方式的描述, 是数据在数据库内部的

表示方式：记录的存储方式（顺序存储，按照 B 树结构存储，按 hash 方法存储），索引的组织方式，数据是否压缩存储，数据是否加密，数据存储记录结构的规定，一个数据库只有一个内模式。

二级映像 DBMS 内部实现这三个抽象层次的联系和转换

外模式 / 模式映像：模式：描述的是数据的全局逻辑结构；外模式：描述的是数据的局部逻辑结构，即全局和部分的关系。

同一个模式可以有任意多个外模式。每一个外模式，数据库系统都有一个外模式 / 模式映像，定义外模式与模式之间的对应关系。映像定义通常包含在各自外模式的描述中。

保证数据的逻辑独立性：当模式改变时，数据库管理员修改有关的外模式 / 模式映像，使外模式保持不变。应用程序是依据数据的外模式编写的，从而应用程序不必修改，保证了数据与程序的逻辑独立性，简称数据的逻辑独立性。

模式 / 内模式映像（唯一的）：模式 / 内模式映像定义了数据全局逻辑结构与存储结构之间的对应关系。存储结构改变不影响数据库的逻辑结构，这称为数据库的物理独立性

保证数据的物理独立性：当数据库的存储结构改变了（例如选用了另一种存储结构），数据库管理员修改模式 / 内模式映像，使模式保持不变。应用程序不受影响。保证了数据与程序的物理独立性，简称数据的物理独立性。

数据库系统的组成：硬件平台及数据库、软件（DBMS，OS，开发工具，应用程序）、人员（数据库管理员 DBA，系统分析员，应用程序员，用户）。

DBA 的职责：数据库内信息的内容和结构；存储结构和存储策略；完整性约束和安全性条件；监控数据库运行；数据库改进和重组

◆ 关系系统主要按什么进行分类？可分为哪几类？

分类标准：结构(Structure)，完整性(Integrity)，数据操纵(Manipulation)

可分为四类：表式系统；最小关系系统；关系完备的系统；全关系系统

◆ 查询优化的具体实现步骤：

把查询转换成某种内部表示——把语法树转换成标准（优化）形式。——选择低层的存取路径。——生成查询计划、选择代价最小的。

第二章 数据模型

模型：数据模型不仅要反应数据本身，还要反应出数据之间的关系

数据模型的要求：真实；易于理解；易于通过计算机实现

数据模型分为两类：

(1) 概念模型：也称信息模型，按用户的观点来对数据和信息建模，用于数据库设计。

(2) 逻辑模型：包括层次模型，网状模型，关系模型。

数据模型的组成要素：数据结构，数据操作，完整性约束条件

数据结构：是所研究对象类型的集合，包含数据内容、数据和数据之间的联系。是数据的静态描述。

数据操作：是对数据库中各个对象允许执行操作的集合以及相关操作的规则(插入，删除，修改)

约束条件：是一组完整性规则的集合，给定数据模型中数据及其联系具有的制约和依存规则

概念模型：现实到信息世界的第一层抽象，数据库设计人员进行数据库设计的工具：

(1) 实体 (Entity)：客观存在并可相互区别的事物称为实体。可以是具体的人、事、物或抽象的概念。

(2) 属性 (Attribute)：实体所具有的某一特性称为属性。

(3)码 (Key) : 唯一标识实体的属性集称为码。

(4)域 (Domain): 属性的取值范围称为该属性的域。

(5)实体型 (Entity Type): 用实体名及其属性名集合来抽象和刻画同类实体

(6)实体集 (Entity Set): 同一类型实体的集合称为实体集

(7)关系 (Relationship): 现实世界中事物内部以及事物之间的联系在信息世界中反映为实体内部的联系和实体之间的联系。

两个实体型之间的联系:

一对一联系 (1:1): 如果对于实体集 A 中的每一个实体, 实体集 B 中至多有一个 (也可以没有) 实体与之联系, 反之亦然, 则称实体集 A 与实体集 B 具有一对一联系, 记为 1:1

一对多联系 (1:n): 如果对于实体集 A 中的每一个实体, 实体集 B 中有 n 个实体 ($n \geq 0$) 与之联系, 反之, 对于实体集 B 中的每一个实体, 实体集 A 中至多只有一个实体与之联系, 则称实体集 A 与实体集 B 有一对多联系, 记为 1:n

多对多联系 (m:n): 如果对于实体集 A 中的每一个实体, 实体集 B 中有 n 个实体 ($n \geq 0$) 与之联系, 反之, 对于实体集 B 中的每一个实体, 实体集 A 中也有 m 个实体 ($m \geq 0$) 与之联系, 则称实体集 A 与实体 B 具有多对多联系, 记为 m:n

概念模型的一种表示方法: 实体-联系方法(E-R 方法)

实体型: 用矩形表示, 矩形框内写明实体名。

属性: 用椭圆形表示, 并用无向边将其与相应的实体连接起来

联系: 用菱形表示, 菱形框内写明联系名, 并用无向边分别与有关实体连接起来, 同时在无向边旁标上联系的类型 (1:1、1:n 或 m:n)

最常用的数据模型: 非关系模型 (指两个记录以及它们之间的联系, 如层次模型(Hierarchical Model)、网状模型(Network Model))、关系模型(Relational Model)、面向对象模型(Object Oriented Model)、对象关系模型(Object Relational Model)

层次结构: 用树型结构表示实体和实体之间的联系; 有且只有一个节点没有双亲节点, 根节点; 其他节点有且仅有一个双亲节点; 有相同双亲节点的节点称为兄弟节点; 没有子女节点的节点称为叶节点; 每个节点是一个数据类型; 每个节点由若干个字段构成

层次模型的数据操纵和约束: 没有双亲不能进行插入操作, 删除双亲则子女节点也同时删除, 修改需要修改所有相应的记录

网状模型: 允许 1 个以上的节点无双亲, 一个节点可以有多个双亲

网状模型的数据操作和完整性约束: 支持记录码的概念, 不允许出现重复值; 保证一个联系中, 双亲记录和子女记录之间是 1 对多的关系; 支持双亲记录和子女记录之间的约束条件

网状模型的优缺点: 优点: 能很好地表示多对多的关系, 性能较高; 缺点: 结构复杂, DDL, DML 语言复杂

关系数据模型的数据结构:

关系 (Relation): 一个关系对应通常说的一张二维表

元组 (Tuple): 表中的一行即为一个元组。

属性 (Attribute): 表中的一列即为一个属性, 给每一个属性起一个名称即属性名。

主码 (Key): 表中的某个属性组, 它可以唯一确定一个元组。

域 (Domain): 属性的取值范围。

分量: 元组中的一个属性值。关系中的每个分量不可再分, 即不允许表中还包含表。

关系模式: 对关系的描述 关系名 (属性 1, 属性 2, 属性 3)

关系模型中数据操纵包括: 查询, 插入, 删除, 和修改, 所有的操纵是针对关系表进行

关系的完整性约束条件: 实体完整性; 参照完整性; 用户定义的完整性。

关系型数据库的优缺点: 优点: 理论基础完善; 模型单一, 无论实体和实体关系统一用表的

形式表示；一体化的数据子语言；存取方式对用户透明，具有更好的数据独立性；面向集合的存取；有利于应用的扩展
缺点：查询效率较低，需要用户对查询进行优化

第三章 关系数据库

单一的数据结构---关系 逻辑结构---二维表

域：是一组具有相同数据类型的值的集合。

笛卡尔积：给定一组域 $D_1, D_2, \dots, D_n$ ，这些域中可以有相同的。

$D_1, D_2, \dots, D_n$ 的笛卡尔积为：

$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_i \in D_i, i = 1, 2, \dots, n\}$ 所有域的所有取值的一个组合，不能重复

元组 (Tuple) 笛卡尔积中每一个元素 $(d_1, d_2, \dots, d_n)$ 叫作一个 n 元组 (n-tuple) 或简称元组(Tuple)

分量 (Component)：笛卡尔积元素 $(d_1, d_2, \dots, d_n)$ 中的每一个值 d_i 叫作一个分量。

基数 (Cardinal number)：若 $D_i (i = 1, 2, \dots, n)$ 为有限集，其基数为 $m_i (i = 1, 2, \dots, n)$ ，则 $D_1 \times D_2 \times \dots \times D_n$ 的基数 M 为：

$$M = \prod_{i=1}^n m_i$$

笛卡尔积的表示方法：笛卡尔积可表示为一个二维表，表中的每行对应一个元组，表中的每列对应一个域。

关系： $D_1 \times D_2 \times \dots \times D_n$ 的子集叫作在域 $D_1, D_2, \dots, D_n$ 上的关系，表示为 $R(D_1, D_2, \dots, D_n)$ R ：关系名 n ：关系的目或度 (Degree)

元组：关系中的每个元素是关系中的元组，通常用 t 表示。

单元关系与二元关系：当 $n=1$ 时，称该关系为单元关系 (Unary relation) 或一元关系当 $n=2$ 时，称该关系为二元关系 (Binary relation)。

关系的表示：关系也是一个二维表，表的每行对应一个元组，表的每列对应一个域

属性：关系中不同列可以对应相同的域，为了加以区分，必须对每列起一个名字，称为属性 (Attribute)， n 目关系必有 n 个属性

码：候选码 (Candidate key)：若关系中的某一属性组的值能唯一地标识一个元组，则称该属性组为候选码。简单的情况：候选码只包含一个属性

全码 (All-key) 最极端的情况：关系模式的所有属性组是这个关系模式的候选码，称为全码 (All-key)

主码：若一个关系有多个候选码，则选定其中一个为主码 (Primary key)

主属性：候选码的诸属性称为主属性 (Prime attribute) 不包含在任何候选码中的属性称为非主属性 (Non-Prime attribute) 或非码属性 (Non-key attribute)

基本关系 (基本表或基表)：实际存在的表，是实际存储数据的逻辑表示

查询表：查询结果对应的表

视图表由基本表或其他视图表导出的表，是虚表，不对应实际存储的数据

基本关系的性质：① 列是同质的 (Homogeneous) ② 不同的列可出自同一个域：其中的每一列称为一个属性，不同的属性要给予不同的属性名③列的顺序无所谓，列的次序可以任意交换④ 任意两个元组的候选码不能相同⑤ 行的顺序无所谓，行的次序可以任意交换⑥ 分量必须取原子值。

关系模式 (Relation Schema) 是型，关系是值，关系模式是对关系的描述。

关系模式可以形式化地表示为： $R(U, D, DOM, F)$

R ——关系名 U —— 组成该关系的属性名集合 D ——属性组 U 中属性所来自的域
 DOM —— 属性向域的映象集合 F —— 属性间的数据依赖关系集合

关系模式：对关系的描述；静态的、稳定的

关系：关系模式在某一时刻的状态或内容；动态的、随时间不断变化的

关系的三类完整性约束：

实体完整性和参照完整性：关系模型必须满足的完整性约束条件，称为关系的两个不变性，应该由关系系统自动支持。

用户定义的完整性：应用领域需要遵循的约束条件，体现了具体领域中的语义约束

实体完整性规则 (Entity Integrity) 若属性 A 是基本关系 R 的主属性，则属性 A 不能取空值。

外码：设 F 是基本关系 R 的一个或一组属性，但不是关系 R 的码。如果 F 与基本关系 S 的主码 K_s 相对应，则称 F 是基本关系 R 的外码。基本关系 R 称为参照关系 (Referencing Relation) 基本关系 S 称为被参照关系 (Referenced Relation) 或目标关系 (Target Relation)。

参照完整性规则：若属性 (或属性组) F 是基本关系 R 的外码它与基本关系 S 的主码 K_s 相对应 (基本关系 R 和 S 不一定是不同的关系)，则对于 R 中每个元组在 F 上的值必须为：或者取空值 (F 的每个属性值均为空值) 或者等于 S 中某个元组的主码值

用户定义的完整性：针对某一具体关系数据库的约束条件，反映某一具体应用所涉及的数据必须满足的语义要求。

传统的集合运算：关系代数：

并 (Union)： $R \cup S$ 仍为 n 目关系，由属于 R 或属于 S 的元组组成：

$$R \cup S = \{ t | t \in R \vee t \in S \}$$

差 (Difference)： $R - S$ 仍为 n 目关系，由属于 R 而不属于 S 的所有元组组成

$$R - S = \{ t | t \in R \wedge t \notin S \}$$

交 (Intersection) $R \cap S$ 仍为 n 目关系，由既属于 R 又属于 S 的元组组成：

$$R \cap S = \{ t | t \in R \wedge t \in S \} \quad R \cap S = R - (R - S)$$

笛卡尔积 (Cartesian Product)： R ： n 目关系， k_1 个元组， S ： m 目关系， k_2 个元组
 $R \times S$ 列： $(n+m)$ 列元组的集合 行： $k_1 \times k_2$ 个元组 $R \times S = \{ tr \ ts | tr \in R \wedge ts \in S \}$

专门的关系运算：

选择 (Selection)：为限制 (Restriction) 含义：在关系 R 中选择满足给定条件的诸元组：
 $\sigma_F(R) = \{ t | t \in R \wedge F(t) = \text{'真'} \}$ F ：选择条件，是一个逻辑表达式，基本形式为： $X_1 \theta Y_1$
选择运算是从关系 R 中选取使逻辑表达式 F 为真的元组，是从行的角度进行的运算

投影 (Projection)：含义从 R 中选择出若干属性列组成新的关系

$\pi_A(R) = \{ t[A] | t \in R \}$ A ： R 中的属性列 投影操作主要是从列的角度进行运算

连接 (Join) 也称为 θ 连接，含义：从两个关系的笛卡尔积中选取属性间满足一定条件的元组：
 $R \bowtie S = \{ tr \ ts | tr \in R \wedge ts \in S \wedge tr[A] \theta ts[B] \}$

A 和 B ：分别为 R 和 S 上度数相等且可比的属性组 θ ：比较运算符

连接运算从 R 和 S 的广义笛卡尔积 $R \times S$ 中选取 (R 关系) 在 A 属性组上的值与 (S 关系) 在 B 属性组上值满足比较关系 θ 的元组。一般的连接操作是从行的角度进行运算

等值连接 (equijoin) θ 为“=”的连接运算称为等值连接：

含义：从关系 R 与 S 的广义笛卡尔积中选取 A 、 B 属性值相等的那些元组，即等值连接为：

$$R \bowtie S = \{ tr \ ts | tr \in R \wedge ts \in S \wedge tr[A] = ts[B] \}$$

自然连接 (Natural join)：是一种特殊的等值连接。两个关系中进行比较的分量必须是相同的属性组。在结果中把重复的属性列去掉

含义： R 和 S 具有相同的属性组 B

$$R \bowtie S = \{ \quad \mid \pi_B R \cap \pi_B S \neq \emptyset \}$$

扩充的关系运算:

属性重命名: 有时候为了实现关系自身和自身之间的运算, 可以将某些属性的名称进行换名操作, 以便可以在同一个关系上进行自然链接等运算

外连接: 如果把舍弃的元组也保存在结果关系中, 而在其他属性上填充值(Null), 这种连接就叫做外连接 (OUTER JOIN)。

左外连接: 如果只把左边关系 R 中要舍弃的元组保留就叫做左外连接(LEFT OUTER JOIN 或 LEFT JOIN)

右外连接: 如果只把右边关系 S 中要舍弃的元组保留就叫做右外连接(RIGHT OUTER JOIN 或 RIGHT JOIN)。

除 (Division): 给定关系 $R(X, Y)$ 和 $S(Y, Z)$, 其中 X, Y, Z 为属性组。 R 中的 Y 与 S 中的 Y 可以有不同属性名, 但必须出自相同的域集。 R 与 S 的除运算得到一个新的关系 $P(X)$, P 是 R 中满足下列条件的元组在 X 属性列上的投影: 元组在 X 上分量值 x 的象集 Y_x 包含 S 在 Y 上投影的集合, 记作:

$$R \div S = \{ \pi_X R \mid \pi_X R \cap \pi_X S \neq \emptyset \} \quad Y_x: x \text{ 在 } R \text{ 中的象集, } x = \pi_X R$$

除操作是同时从行和列角度进行运算

关系演算: 关系演算来源于数理逻辑的谓词演算, 按照变元不同, 分为元组关系演算和域关系演算, 一般的用法是使用谓词给出查询结果需要满足的条件, 系统完成查询操作, 高度非过程化 (关系代数需要指明运算顺序)

第四章关系数据库标准语言 SQL

SQL 的特点: 综合统一; 高度非过程化; 面向集合的操作方式; 以同一种语法结构提供多种使用方式; 语言简洁, 易学易用;

基本表: 本身独立存在的表; SQL 中一个关系就对应一个基本表; 一个(或多个)基本表对应一个存储文件; 一个表可以带若干索引

存储文件: 逻辑结构组成了关系数据库的内模式; 物理结构是任意的, 对用户透明

视图: 从一个或几个基本表导出的表; 数据库中只存放视图的定义而不存放视图对应的数据; 视图是一个虚表; 用户可以在视图上再定义视图

定义模式: CREATE SCHEMA <模式名> AUTHORIZATION <用户名> [<表定义子句> | <视图定义子句> | <授权定义子句>]

删除模式: DROP SCHEMA <模式名> [<CASCADE> | <RESTRICT>] CASCADE (级联)

基本表操作:

定义基本表:

CREATE TABLE <表名> (<列名> <数据类型> [<列级完整性约束条件>]

[, <列名> <数据类型> [<列级完整性约束条件>]] ... [, <表级完整性约束条件>])

每一个基本表都属于某一个模式, 一个模式包含多个基本表, 定义基本表所属模式

修改基本表:

ALTER TABLE <表名>

[ADD <新列名> <数据类型> [完整性约束]]

[DROP <完整性约束名>]

[ALTER COLUMN <列名> <数据类型>];

删除基本表 :

DROP TABLE <表名> [RESTRICT | CASCADE];

建立索引：

CREATE [UNIQUE] [CLUSTER] INDEX <索引名>

ON <表名>(<列名>[<次序>][,<列名>[<次序>]]...);

在最经常查询的列上建立聚簇索引以提高查询效率，一个基本表上最多只能建立一个聚簇索引，经常更新的列不宜建立聚簇索引。

删除索引： DROP INDEX <索引名>;

查询满足条件的元组：

谓词：(NOT) BETWEEN ... AND ...

谓词：IN <值表>, NOT IN <值表>

谓词：[NOT] LIKE ‘<匹配串>’ [ESCAPE ‘<换码字符>’]

逻辑运算符：AND 和 OR 来联结多个查询条件

ORDER BY 子句： 升序：ASC；降序：DESC；缺省值为升序

聚集函数：

计数：COUNT ([DISTINCT|ALL] *) COUNT ([DISTINCT|ALL] <列名>)

计算总和：SUM ([DISTINCT|ALL] <列名>)

计算平均值：AVG ([DISTINCT|ALL] <列名>)

最大最小值：MAX ([DISTINCT|ALL] <列名>) MIN ([DISTINCT|ALL] <列名>)

GROUP BY 子句：分组

HAVING 短语与 WHERE 子句的区别：

- 作用对象不同
- WHERE 子句作用于基表或视图，从中选择满足条件的元组
- HAVING 短语作用于组，从中选择满足条件的组。

连接查询：

等值连接：连接运算符为=

自身连接：一个表与其自己进行连接，需要给表起别名以示区别，由于所有属性名都是同名属性，因此必须使用别名前缀

外连接 左外连接：列出左边关系（如本例 Student）中所有的元组

右外连接：列出右边关系中所有的元组

复合条件连接： WHERE 子句中含多个连接条件

嵌套查询： 一个 SELECT-FROM-WHERE 语句称为一个查询块；将一个查询块嵌套在另一个查询块的 WHERE 子句或 HAVING 短语的条件中的查询称为嵌套查询。子查询的限制：不能使用 ORDER BY 子句

集合操作的种类： 并操作 UNION；交操作 INTERSECT；差操作 EXCEPT

数据处理：

数据查询：

SELECT [ALL|DISTINCT] <目标列表达式>[, <目标列表达式>] ...

FROM <表名或视图名>[, <表名或视图名>] ...

[WHERE <条件表达式>]

[GROUP BY <列名 1> [HAVING <条件表达式>]]

[ORDER BY <列名 2> [ASC|DESC]];

插入元组

INSERT

INTO <表名> [(<属性列 1>[, <属性列 2>...])

VALUES (<常量 1>[, <常量 2>] ...)

插入子查询结果

INSERT
INTO <表名> [(<属性列 1> [, <属性列 2>...]) SELECT 子查询;

修改数据

UPDATE <表名>
SET <列名>=<表达式>[, <列名>=<表达式>]...
[WHERE <条件>];

删除数据

DELETE
FROM <表名>
[WHERE <条件>];

视图:

视图的特点: 虚表, 是从一个或几个基本表 (或视图) 导出的表, 只存放视图的定义, 不存放视图对应的数据, 基表中的数据发生变化, 从视图中查询出的数据也随之改变

建立视图: CREATE VIEW
<视图名> [(<列名> [, <列名>]...)]
AS <子查询>
[WITH CHECK OPTION]

删除视图: DROP VIEW <视图名>;

视图的作用:

1. 视图能够简化用户的操作
2. 视图使用户能以多种角度看待同一数据
3. 视图对重构数据库提供了一定程度的逻辑独立性
4. 视图能够对机密数据提供安全保护
5. 适当的利用视图可以更清晰的表达查询

数据控制: 数据库系统可以由 DBMS 统一提供授权控制

- ◆ 事务管理功能 – 事务, 提交, 回滚
- ◆ 数据保护功能 – 完整性约束, 安全控制功能

安全控制功能: 什么人对什么数据有什么样的访问能力 – 不是技术问题, 是安全策略问题
DBMS 需要对保障安全策略的实施

DBMS 需要具备的安全功能:

授权决定的设置 – SQL 中的 GRANT 和 REVOKE

授权结果保存在数据字典

当用户提出数据访问请求时候, 根据数据字典决定是否执行操作请求

授权: Grant **收回授权:** Revoke, 授权收回会级联下去。

授权与回收:

GRANT 语句的一般格式:
GRANT <权限>[,<权限>]...
[ON <对象类型> <对象名>]
TO <用户>[,<用户>]...
[WITH GRANT OPTION];

REVOKE 语句的一般格式为:
REVOKE <权限>[,<权限>]...
[ON <对象类型> <对象名>]
FROM <用户>[,<用户>]...;

创建数据库模式的权限

CREATE USER 语句格式: CREATE USER <username>
[WITH] [DBA | RESOURCE | CONNECT]

用户对自己建立的表拥有全部权限，并且可以授权

数据库角色

角色的创建： CREATE ROLE <角色名>

给角色授权：

GRANT <权限> [, <权限>] ...

ON <对象类型>对象名

TO <角色> [, <角色>] ...

将一个角色授予其他的角色或用户：

GRANT <角色 1> [, <角色 2>] ...

TO <角色 3> [, <用户 1>] ...

[WITH ADMIN OPTION]

角色权限的收回：

REVOKE <权限> [, <权限>] ...

ON <对象类型> <对象名>

FROM <角色> [, <角色>] ...

嵌入式 SQL:

- ◆ 交互的 – 面向集合的 – 非过程性
- ◆ 嵌入式 – 将 SQL 语言嵌入到其他高级语言中
- ◆ 该高级语言称为主语言或者宿主语言
- ◆ 宿主形式和交互形式在细节上会有所不同 – 虽然语法形式相同。

嵌入式 SQL 的一般形式:

- ◆ 预编译 – DBMS 对源程序进行预扫描 – 识别出 SQL 语句，转换成主语言的调用 – 主语言编译程序转换成目标码 – 这种形式用的比较多
- ◆ 扩充主语言
- ◆ 一般形式 SQL 语句前加 EXEC SQL 字头，后面跟 END – EXEC 或者主语言的分割符

SQL 通讯区 SQLCA:

- ◆ SQLCA 实际是一个数据结构，通过 exec SQL INLCUDE SQLCA 来定义
- ◆ SQL 语句执行后，执行情况和状态会保存在这个数据结构中
- ◆ 应用程序在执行结束后应该检测 SQLCA 中的状态以便决定下一步处理的流程

主变量:

- ◆ SQL 语句中使用的主语言的变量称为主变量
- ◆ 输入主变量 – 应用程序赋值，SQL 语句引用
- ◆ 输出主变量 – SQL 语句赋值，反馈给主程序
- ◆ 主变量可以附带一个指示变量用以标明主变量的状态。
- ◆ 主变量的声明必须在 BEGIN DECLERE SECTION 和 END DECLERE SECTION 之间，在 SQL 语句中需要前缀：以区别数据库对象名称，指示变量也需要前面加前缀 (:) 并且在主变量之后
- ◆ 在主语言中，主变量可以随意引用，不必加：

游标:

- ◆ SQL 语句是面向集合的
- ◆ 主语言是面向记录的
- ◆ 游标是系统开设的一个缓冲区存储 SQL 语句的执行结果，以便主语言顺次访问记录

- ◆ 每个游标都有一个名字

Current 形式的 update 和 delete:

- ◆ 声明游标 在 select 语句中使用子句
For update of<列名>
- ◆ Open 打开游标
- ◆ Fetch 移动游标并返回值到主变量
- ◆ 检查当前记录并执行相应操作 使用
Where current of <游标名>
- ◆ 关闭游标

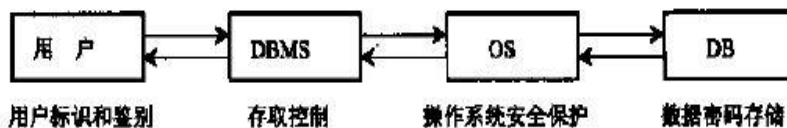
动态 SQL 简介:

- ◆ 静态 SQL 语句 – 内嵌 SQL 语句的变量个数和数据类型在预编译之前是确定的
- ◆ 为了使内嵌 SQL 语句更加灵活, 有些 DBMS 支持动态的 SQL 技术
SQL 语句正文
主变量的个数
主变量的数据类型
SQL 语句引用的数据库对象

第六章 数据库安全性

三类计算机系统安全性问题: 计算机系统安全类, 管理安全类, 政策法律类

数据库的安全性控制: 计算机系统中, 安全措施是一级一级的层层设置的



存取控制: 什么样的用户对什么样的信息有什么访问权限的问题。定义用户权限, 登记到数据字典当中

合法权限的检查机制:

自主存取控制 (DAC) – 用户对不同的数据对象有不同的存取权限。不同的用户对相同的数据也有不同的存取权限

强制存取控制 (MAC) – 每个数据对象被标以一个密级, 某个用户被授予一个级别的访问许可

DBMS 实现 MAC 首先必须实现 DAC

数据库安全性

规则 1: 任何查询至少要涉及 $N(N \text{ 足够大})$ 个以上的记录

规则 2: 任意两个查询的相交数据项不能超过 M 个

规则 3: 任一用户的查询次数不能超过 $1+(N-2)/M$

视图机制: 通过视图机制可以保证一定程度的安全性

审计: 对数据库的所有访问操作记录在审计日志

数据加密: 将数据库中保存的信息从明文加密成密文

数据库统计功能安全性: 防止用户使用数据库统计 (聚合函数) 获得不被授权的单记录信息

第七章 数据库完整性

数据库的完整性: 数据的正确性、有效性和相容性

数据的完整性和安全性是两个不同概念

数据的完整性：防止数据库中存在不符合语义的数据，也就是防止数据库中存在不正确的数据，防范对象：不合语义的、不正确的数据

数据的安全性：保护数据库防止恶意的破坏和非法的存取，防范对象：非法用户和非法操作

完整性约束条件：

约束条件的对象

- 关系 – 元组间，元组集合间或者关系间的约束
- 元组 – 元组的各列之间的约束关系
- 列 – 字段类型，取值范围，精度等

约束的状态

- 静态 – 某个确定的状态时数据的约束条件
- 动态 – 从一个状态到另外一个状态时新旧值之间的约束条件

为维护数据库的完整性，DBMS 必须：

- 1.提供定义完整性约束条件的机制；
- 2.提供完整性检查的方法；
- 3.违约处理

实体完整性定义：关系模型的实体完整性 CREATE TABLE 中用 PRIMARY KEY 定义

实体完整性检查和违约处理：

插入或对主码列进行更新操作时，RDBMS 按照实体完整性规则自动进行检查。包括：

1. 检查主码值是否唯一，如果不唯一则拒绝插入或修改
2. 检查主码的各个属性是否为空，只要有一个为空就拒绝插入或修改

参照完整性定义：在 CREATE TABLE 中用 FOREIGN KEY 短语定义哪些列为外码。

用 REFERENCES 短语指明这些外码参照哪些表的主码。

参照完整性违约处理：

拒绝(NO ACTION)执行 2. 级联(CASCADE)操作 3. 设置为空值 (SET-NULL)

用户定义的完整性就是针对某一具体应用的数据必须满足的语义要求

属性上的约束条件的定义：

CREATE TABLE 时定义：列值非空 (NOT NULL) 列值唯一 (UNIQUE) 检查列值是否满足一个布尔表达式 (CHECK)

属性上的约束条件检查和违约处理：插入元组或修改属性的值时，RDBMS 检查属性上的约束条件是否被满足；如果不满足则操作被拒绝执行

域中的完整性限制：SQL 支持域的概念，并且可以用 CREATE DOMAIN 语句建立一个域以及该域应该满足的完整性约束条件。

触发器——触发器 (Trigger) 是用户定义在关系表上的一类由事件驱动的特殊过程

- 由服务器自动激活
- 可以进行更为复杂的检查和操作，具有更精细和更强大的数据控制能力

定义触发器

```
CREATE TRIGGER <触发器名>
{BEFORE | AFTER} <触发事件> ON <表名>
FOR EACH {ROW | STATEMENT}
[WHEN <触发条件>] <触发动作体>
```

激活触发器：触发器的执行，是由触发事件激活的，并由数据库服务器自动执行一个数据表上可能定义了多个触发器

同一个表上的多个触发器激活时遵循如下的执行顺序：

- (1) 执行该表上的 BEFORE 触发器；

- (2) 激活触发器的 SQL 语句;
- (3) 执行该表上的 AFTER 触发器。

删除触发器: DROP TRIGGER <触发器名> ON <表名>;

- 触发器必须是一个已经创建的触发器, 并且只能由具有相应权限的用户删除。

第八章 数据库恢复技术

事务概念: 用户定义的一个数据库操作序列, 这些操作要么全做要么全不做, 是一个不可分割的工作单位。恢复和并发控制的基本单位。

事务和程序比较: 在关系数据库中, 一个事务可以是一条或多条 SQL 语句, 也可以包含一个或多个程序。一个程序通常包含多个事务

定义事务: 显式定义方式

BEGIN TRANSACTION

SQL 语句 1

SQL 语句 2

.....

COMMIT

BEGIN TRANSACTION

SQL 语句 1

SQL 语句 2

.....

ROLLBACK

隐式方式: 当用户没有显式地定义事务时, DBMS 按缺省规定自动划分事务

事务状态: 活动状态, 部分提交状态, 失败状态, 提交状态, 终止状态。

事务操作原因:

事务 ACID 特征: 原子性 (Atomicity) 一致性 (Consistency) 隔离性 (Isolation) 持续性 (Durability)

故障是不可避免的

系统故障: 计算机软、硬件故障

人为故障: 操作员的失误、恶意的破坏等

故障的种类: 事务内部的故障, 系统故障, 介质故障, 计算机病毒

数据库的恢复: 把数据库从错误状态恢复到某一已知的正确状态(亦称为一致状态或完整状态)

事务内部的故障更多的是非预期的, 是不能由应用程序处理的。

- 运算溢出
- 并发事务发生死锁而被选中撤销该事务
- 违反了某些完整性限制等

以后, 事务故障仅指这类非预期的故障

事务故障的恢复: 撤消事务 (UNDO)

系统故障

称为软故障, 是指造成系统停止运转的任何事件, 使得系统要重新启动。

- 整个系统的正常运行突然被破坏
- 所有正在运行的事务都非正常终止
- 不破坏数据库
- 内存中数据库缓冲区的信息全部丢失

系统故障的常见原因: 特定类型的硬件错误 (如 CPU 故障); 操作系统故障; DBMS 代码错误; 系统断电

系统故障的恢复:

发生系统故障时, 事务未提交

恢复策略：强行撤消 (UNDO) 所有未完成事务

系统故障时，事务已提交，但缓冲区中的信息尚未完全写回到磁盘上。

恢复策略：重做 (REDO) 所有已提交的事务

介质故障

称为硬故障，指外存故障

- 磁盘损坏
- 磁头碰撞
- 操作系统的某种潜在错误
- 瞬时强磁场干扰

介质故障的恢复：

装入数据库发生介质故障前某个时刻的数据副本

重做自此时始的所有成功事务，将这些事务已提交的结果重新记入数据库

计算机病毒：一种人为的故障或破坏，是一些恶作剧者研制的一种计算机程序，可以繁殖和传播

危害：破坏、盗窃系统中的数据，破坏系统文件

各类故障，对数据库的影响有两种可能性

一是数据库本身被破坏

二是数据库没有被破坏，但数据可能不正确，这是由于事务的运行被非正常终止造成的。

恢复操作的基本原理：冗余

利用存储在系统其它地方的冗余数据来重建数据库中已被破坏或不正确的那部分数据

恢复机制涉及的关键问题

1. 如何建立冗余数据

数据转储 (backup) / 登录日志文件 (logging)

2. 如何利用这些冗余数据实施数据库恢复

数据转储：转储是指 DBA 将整个数据库复制到磁带或另一个磁盘上保存起来的过程，备用的数据称为后备副本或后援副本

如何使用

数据库遭到破坏后可以将后备副本重新装入

重装后备副本只能将数据库恢复到转储时的状态

静态转储：在系统中无运行事务时进行的转储操作，转储开始时数据库处于一致性状态，转储期间不允许对数据库的任何存取、修改活动，得到的一定是一个数据一致性的副本。

优点：实现简单 **缺点：**降低了数据库的可用性，转储必须等待正运行的用户事务结束，新的事务必须等转储结束

动态转储：

- 转储操作与用户事务并发进行，转储期间允许对数据库进行存取或修改
- 优点
 - 不用等待正在运行的用户事务结束，不会影响新事务的运行
- 动态转储的缺点
 - 不能保证副本中的数据正确有效
- 利用动态转储得到的副本进行故障恢复
 - 需要把动态转储期间各事务对数据库的修改活动登记下来，建立日志文件
 - 后备副本加上日志文件才能把数据库恢复到某一时刻的正确状态

海量转储：每次转储全部数据库

增量转储：只转储上次转储后更新过的数据

海量转储与增量转储比较

- 从恢复角度看，使用海量转储得到的后备副本进行恢复往往更方便
- 但如果数据库很大，事务处理又十分频繁，则增量转储方式更实用更有效

		转储状态	
		动态转储	静态转储
转储方式	海量转储	动态海量转储	静态海量转储
	增量转储	动态增量转储	静态增量转储

日志文件(log)是用来记录事务对数据库的更新操作的文件

日志文件的格式

■ 以记录为单位的日志文件

各个事务的开始标记(BEGIN TRANSACTION)

各个事务的结束标记(COMMIT 或 ROLLBACK)

各个事务的所有更新操作

以上均作为日志文件中的一个日志记录 (log record)

- 事务标识 (标明是哪个事务)
- 操作类型 (插入、删除或修改)
- 操作对象 (记录内部标识)
- 更新前数据的旧值 (对插入操作而言，此项为空值)
- 更新后数据的新值 (对删除操作而言，此项为空值)

■ 以数据块为单位的日志文件

事务标识 (标明是哪个事务)

被更新的数据块

日志文件的作用:

- 进行事务故障恢复
- 进行系统故障恢复
- 协助后备副本进行介质故障恢复

PPT32, 33

登记日志文件的基本原则:

- 登记的次序严格按并行事务执行的时间次序
- 必须先写日志文件，后写数据库
 - 写日志文件操作: 把表示这个修改的日志记录写到日志文件
 - 写数据库操作: 把对数据的修改写到数据库中

为什么要先写日志文件

- 写数据库和写日志文件是两个不同的操作
- 在这两个操作之间可能发生故障
- 如果先写了数据库修改，而在日志文件中没有登记下这个修改，则以后就无法恢复这个修改了
- 如果先写日志，但没有修改数据库，按日志文件恢复时只不过是多执行一次不必要的 UNDO 操作，并不会影响数据库的正确性

事务故障: 事务在运行至正常终止点前被终止

恢复方法: 由恢复子系统应利用日志文件撤消 (UNDO) 此事务已对数据库进行的修改

事务故障的恢复由系统自动完成，对用户是透明的，不需要用户干预

事务故障的恢复步骤

1. 反向扫描文件日志（即从最后向前扫描日志文件），查找该事务的更新操作。
2. 对该事务的更新操作执行逆操作。即将日志记录中“更新前的值”写入数据库。
 - 插入操作，“更新前的值”为空，则相当于做删除操作
 - 删除操作，“更新后的值”为空，则相当于做插入操作
 - 若是修改操作，则相当于用修改前值代替修改后值
3. 继续反向扫描日志文件，查找该事务的其他更新操作，并做同样处理。
4. 如此处理下去，直至读到此事务的开始标记，事务故障恢复就完成了。

系统故障造成数据库不一致状态的原因

- 未完成事务对数据库的更新已写入数据库
- 已提交事务对数据库的更新还留在缓冲区来不及写入数据库

恢复方法

- 1. Undo 故障发生时未完成的事务
- 2. Redo 已完成的事务

系统故障的恢复由系统在重新启动时自动完成，不需要用户干预

系统故障的恢复步骤

1. 正向扫描日志文件（即从头扫描日志文件）

重做(REDO) 队列: 在故障发生前已经提交的事务

这些事务既有 BEGIN TRANSACTION 记录，也有 COMMIT 记录

撤销 (Undo)队列:故障发生时未完成的事务

这些事务只有 BEGIN TRANSACTION 记录，无相应的 COMMIT 记录

2. 对撤销(Undo)队列事务进行撤销(UNDO)处理

- 反向扫描日志文件，对每个 UNDO 事务的更新操作执行逆操作
- 即将日志记录中“更新前的值”写入数据库

3. 对重做(Redo)队列事务进行重做(REDO)处理

- 正向扫描日志文件，对每个 REDO 事务重新执行登记的操作
- 即将日志记录中“更新后的值”写入数据库

介质故障的恢复: 1.重装数据库 2.重做已经完成的事务

恢复步骤

1. 装入最新的后备数据库副本(离故障发生时刻最近的转储副本)，使数据库恢复到最近一次转储时的一致性状态。

- a) 对于静态转储的数据库副本，装入后数据库即处于一致性状态
- b) 对于动态转储的数据库副本，还须同时装入转储时刻的日志文件副本，利用与恢复系统故障的方法（即 REDO+UNDO），才能将数据库恢复到一致性状态。

2. 装入有关的日志文件副本(转储结束时刻的日志文件副本)，重做已完成的事务。

- a) 首先扫描日志文件，找出故障发生时已提交的事务的标识，将其记入重做队列。
- b) 然后正向扫描日志文件，对重做队列中的所有事务进行重做处理。即将日志记录中“更新后的值”写入数据库。

介质故障的恢复需要 DBA 介入，具体的恢复操作仍由 DBMS 完成

- DBA 的工作
 - 重装最近转储的数据库副本和有关的各日志文件副本
 - 执行系统提供的恢复命令

检查点技术: 两个问题:

- 搜索整个日志将耗费大量的时间
- REDO 处理：重新执行，浪费了大量时间
- 解决方案：具有检查点（checkpoint）的恢复技术**
- 在日志文件中增加检查点记录（checkpoint）
- 增加重新开始文件
- 恢复子系统在登录日志文件期间动态地维护日志

检查点记录的内容

- 1. 建立检查点时刻所有正在执行的事务清单
- 2. 这些事务最近一个日志记录的地址

重新开始文件的内容：记录各个检查点记录在日志文件中的地址

动态维护日志文件的方法

周期性地执行如下操作：建立检查点，保存数据库状态。具体步骤是：

- 1.将当前日志缓冲区中的所有日志记录写入磁盘的日志文件上
- 2.在日志文件中写入一个检查点记录
- 3.将当前数据缓冲区的所有数据记录写入磁盘的数据库中
- 4.把检查点记录在日志文件中的地址写入一个重新开始文件

恢复子系统可以定期或不定期地建立检查点,保存数据库状态

■ 定期

按照预定的一个时间间隔，如每隔一小时建立一个检查点

■ 不定期

按照某种规则，如日志文件已写满一半建立一个检查点

使用检查点方法可以改善恢复效率

利用检查点的恢复步骤：

- 1.从重新开始文件中找到最后一个检查点记录在日志文件中的地址，由该地址在日志文件中找到最后一个检查点记录
- 2.由该检查点记录得到检查点建立时刻所有正在执行的事务清单 ACTIVE-LIST
 - a) 建立两个事务队列
 - UNDO-LIST
 - REDO-LIST
 - b) 把 ACTIVE-LIST 暂时放入 UNDO-LIST 队列，REDO 队列暂为空。
- 3.从检查点开始正向扫描日志文件，直到日志文件结束
 - 如有新开始的事务 T_i ，把 T_i 暂时放入 UNDO-LIST 队列
 - 如有提交的事务 T_j ，把 T_j 从 UNDO-LIST 队列移到 REDO-LIST 队列
- 4.对 UNDO-LIST 中的每个事务执行 UNDO 操作
对 REDO-LIST 中的每个事务执行 REDO 操作

数据库镜像

- DBMS 自动把整个数据库或其中的关键数据复制到另一个磁盘上
- DBMS 自动保证镜像数据与主数据库的一致性
每当主数据库更新时，DBMS 自动把更新后的数据复制过去
- **出现介质故障时**
 - 可由镜像磁盘继续提供使用
 - 同时 DBMS 自动利用镜像磁盘数据进行数据库的恢复
 - 不需要关闭系统和重装数据库副本
- **没有出现故障时**

- 可用于并发操作
- 一个用户对数据加排他锁修改数据，其他用户可以读镜像数据库上的数据，而不必等待该用户释放锁
- **频繁地复制数据自然会降低系统运行效率**
 - 在实际应用中用户往往只选择对关键数据和日志文件镜像，而不是对整个数据库进行镜像

第九章 并发控制

多用户控制系统允许多个用户同时使用数据库，导致同一时刻并发的事务很多

不同的多事务执行方式：

(1)事务串行执行

- 每个时刻只有一个事务运行，其他事务必须等到这个事务结束以后方能运行
- 不能充分利用系统资源，发挥数据库共享资源的特点

(2)交叉并发方式 (Interleaved Concurrency)

- 在单处理机系统中，事务的并行执行是这些并行事务的并行操作轮流交叉运行
- 单处理机系统中的并行事务并没有真正地并行运行，但能够减少处理机的空闲时间，提高系统的效率

(3)同时并发方式 (simultaneous concurrency)

- 多处理机系统中，每个处理机可以运行一个事务，多个处理机可以同时运行多个事务，实现多个事务真正的并行运行

事务并发执行带来的问题

- 会产生多个事务同时存取同一数据的情况
- 可能会存取和存储不正确的数据，破坏事务一致性和数据库的一致性

并发控制机制的任务

- 对并发操作进行正确调度
- 保证事务的隔离性
- 保证数据库的一致性

并发操作带来的数据不一致性——由于并发操作破坏了事务的隔离性

- 丢失修改 (Lost Update)
 - 两个事务 T_1 和 T_2 读入同一数据并修改， T_2 的提交结果破坏了 T_1 提交的结果，导致 T_1 的修改被丢失。
- 不可重复读 (Non-repeatable Read)
 - 不可重复读是指事务 T_1 读取数据后，事务 T_2 执行更新操作，使 T_1 无法再现前一次读取结果。
 - 不可重复读包括三种情况：
 - (1)事务 T_1 读取某一数据后，事务 T_2 对其做了修改，当事务 T_1 再次读该数据时，得到与前一次不同的值
 - (2)事务 T_1 按一定条件从数据库中读取了某些数据记录后，事务 T_2 删除了其中部分记录，当 T_1 再次按相同条件读取数据时，发现某些记录消失了
 - (3)事务 T_1 按一定条件从数据库中读取某些数据记录后，事务 T_2 插入了一些记录，当 T_1 再次按相同条件读取数据时，发现多了一些记录。
 - 后两种不可重复读有时也称为幻影现象 (Phantom Row)
- 读“脏”数据 (Dirty Read)

读“脏”数据是指:

- 事务 T1 修改某一数据, 并将其写回磁盘
- 事务 T2 读取同一数据后, T1 由于某种原因被撤销
- 这时 T1 已修改过的数据恢复原值, T2 读到的数据就与数据库中的数据不一致
- T2 读到的数据就为“脏”数据, 即不正确的数据

记号 R(x):读数据 x W(x):写数据 x

并发控制就是要用正确的方式调度并发操作, 使一个用户事务的执行不受其他事务的干扰, 从而避免造成数据的不一致性

• **并发控制的主要技术**

- 有封锁(Locking) 时间戳(Timestamp) 乐观控制法
- 商用的 DBMS 一般都采用封锁方法

封锁:

- 封锁就是事务 T 在对某个数据对象 (例如表、记录等) 操作之前, 先向系统发出请求, 对其加锁
- 加锁后事务 T 就对该数据对象有了一定的控制, 在事务 T 释放它的锁之前, 其它的事务不能更新此数据对象
- 基本封锁类型
 - 排它锁 (Exclusive Locks, 简记为 X 锁)
 - 排它锁又称为写锁
 - 若事务 T 对数据对象 A 加上 X 锁, 则只允许 T 读取和修改 A, 其它任何事务都不能再对 A 加任何类型的锁, 直到 T 释放 A 上的锁
 - 保证其他事务在 T 释放 A 上的锁之前不能再读取和修改 A
 - 共享锁 (Share Locks, 简记为 S 锁)
 - 共享锁又称为读锁
 - 若事务 T 对数据对象 A 加上 S 锁, 则其它事务只能再对 A 加 S 锁, 而不能加 X 锁, 直到 T 释放 A 上的 S 锁
 - 保证其他事务可以读 A, 但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改

锁的相容矩阵

$\begin{matrix} T_2 \\ T_1 \end{matrix}$	X	S	-
X	N	N	Y
S	N	Y	Y
-	Y	Y	Y

在锁的相容矩阵中:

- 最左边一列表示事务T1已经获得的数据对象上的锁的类型, 其中横线表示没有加锁。
- 最上面一行表示另一事务T2对同一数据对象发出的封锁请求。
- T2的封锁请求能否被满足用矩阵中的Y和N表示
 - Y表示事务T2的封锁要求与T1已持有的锁相容, 封锁请求可以满足
 - N表示T2的封锁请求与T1已持有的锁冲突, T2的请求被拒绝

Y=Yes, 相容的请求

N=No, 不相容的请求

封锁协议:

- 如何申请锁 持有锁 释放锁的规则 称为封锁协议
- **一级封锁协议:**事务中对数据进行修改之前必须对其加排他锁直到事务结束才释放该锁
一级封锁协议 可有效防止丢失更新
- **二级封锁协议:**在一级封锁协议基础上, 事务中读取数据之前必须对数据加共享锁, 读完后即可释放该锁。
可以有效防止读脏数据问题

- **三级封锁协议**：在二级封锁协议基础上 增加某事务施加的共享锁保持到事务结束时才释放

活锁和死锁 封锁技术可以有效地解决并行操作的一致性问题，但也带来一些新的问题

活锁：

- 事务 T1 封锁了数据 R
- 事务 T2 又请求封锁 R，于是 T2 等待。
- T3 也请求封锁 R，当 T1 释放了 R 上的封锁之后系统首先批准了 T3 的请求，T2 仍然等待。
- T4 又请求封锁 R，当 T3 释放了 R 上的封锁之后系统又批准了 T4 的请求……
- T2 有可能永远等待，这就是活锁的情形

避免活锁：采用先来先服务 FIFO 的策略

- 当多个事务请求封锁同一数据对象时
- 按请求封锁的先后次序对这些事务排队
- 该数据对象上的锁一旦释放，首先批准申请队列中第一个事务获得锁

死锁：

- 事务 T1 封锁了数据 R1
- T2 封锁了数据 R2
- T1 又请求封锁 R2，因 T2 已封锁了 R2，于是 T1 等待 T2 释放 R2 上的锁
- 接着 T2 又申请封锁 R1，因 T1 已封锁了 R1，T2 也只能等待 T1 释放 R1 上的锁
- 这样 T1 在等待 T2 而 T2 又在等待 T1，T1 和 T2 两个事务永远不能结束，形成死锁

1. 死锁的预防

- 产生死锁的原因是两个或多个事务都已封锁了一些数据对象，然后又都请求对已为其他事务封锁的数据对象加锁，从而出现死等待。
- 预防死锁的发生就是要破坏产生死锁的条件

预防死锁的方法

- 一次封锁法
 - 要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行
- 存在的问题
 - 降低系统并发度
 - 难于事先精确确定封锁对象
- 顺序封锁法
 - 预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁。
- 存在的问题
 - 维护成本高：数据库系统中封锁的数据对象极多，并且在不断地变化。
 - 难以实现：很难事先确定每一个事务要封锁哪些对象

结论

- 在操作系统中广为采用的预防死锁的策略并不很适合数据库的特点
- DBMS 在解决死锁的问题上更普遍采用的是**诊断并解除死锁**的方法

2. 死锁的诊断与解除

死锁的诊断

■ 超时法

- 如果一个事务的等待时间超过了规定的时限，就认为发生了死锁
- 优点：实现简单
- 缺点：有可能误判死锁；时限若设置得太长，死锁发生后不能及时发现

■ 事务等待图法

- 用事务等待图动态反映所有事务的等待情况
事务等待图是一个有向图 $G=(T, U)$
 T 为结点的集合, 每个结点表示正运行的事务
 U 为边的集合, 每条边表示事务等待的情况
若 T_1 等待 T_2 , 则 T_1, T_2 之间划一条有向边, 从 T_1 指向 T_2
- 并发控制子系统周期性地 (比如每隔数秒) 生成事务等待图, 检测事务。如果发现图中存在回路, 则表示系统中出现了死锁。

解除死锁

- 选择一个处理死锁代价最小的事务, 将其撤消
- 释放此事务持有的所有的锁, 使其它事务能继续运行下去

可串行化(Serializable)调度

多个事务的并发执行是正确的, 当且仅当其结果与按某一次序串行地执行这些事务时的结果相同

可串行性(Serializability)

是并发事务正确调度的准则, 一个给定的并发调度, 当且仅当它是可串行化的, 才认为是正确调度

可串行化调度的充分条件

- 一个调度 Sc 在保证冲突操作的次序不变的情况下, 通过交换两个事务不冲突操作的次序得到另一个调度 Sc' , 如果 Sc' 是串行的, 称调度 Sc 为冲突可串行化的调度
- 一个调度是冲突可串行化, 一定是可串行化的调度

冲突操作

- 冲突操作是指不同的事务对同一个数据的读写操作和写写操作
 - $R_i(x)$ 与 $W_j(x)$ /* 事务 T_i 读 x , T_j 写 x */
 - $W_i(x)$ 与 $W_j(x)$ /* 事务 T_i 写 x , T_j 写 x */
- 其他操作是不冲突操作
- 不同事务的冲突操作和同一事务的两个操作不能交换(Swap)

冲突可串行化调度是可串行化调度的充分条件, 不是必要条件。还有不满足冲突可串行化条件的可串行化调度

两段锁协议(Two-Phase Locking, 简称 2PL)是最常用的一种封锁协议, 理论上证明使用两段封锁协议产生的是可串行化调度

• 两段锁协议

指所有事务必须分两个阶段对数据项加锁和解锁

- 在对任何数据进行读、写操作之前, 事务首先要获得对该数据的封锁
- 在释放一个封锁之后, 事务不再申请和获得任何其他封锁

• “两段”锁的含义——事务分为两个阶段

- 第一阶段是获得封锁, 也称为扩展阶段
 - 事务可以申请获得任何数据项上的任何类型的锁, 但是不能释放任何锁
- 第二阶段是释放封锁, 也称为收缩阶段
 - 事务可以释放任何数据项上的任何类型的锁, 但是不能再申请任何锁

• 事务遵守两段锁协议是可串行化调度的充分条件, 而不是必要条件。

• 若并发事务都遵守两段锁协议, 则对这些事务的任何并发调度策略都是可串行化的

• 若并发事务的一个调度是可串行化的, 不一定所有事务都符合两段锁协议

两段锁协议与防止死锁的一次封锁法

一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行，因此一次封锁法遵守两段锁协议

但是两段锁协议并不要求事务必须一次将所有要使用的数据全部加锁，因此遵守两段锁协议的事务可能发生死锁

封锁粒度：封锁对象的大小称为封锁粒度(Granularity)

- 封锁的对象：逻辑单元，物理单元
例：在关系数据库中，封锁对象：
 - 逻辑单元：属性值、属性值集合、元组、关系、索引项、整个索引、整个数据库等
 - 物理单元：页（数据页或索引页）、物理记录等
- **封锁粒度与系统的并发度和并发控制的开销密切相关。**
 - 封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；
 - 封锁的粒度越小，并发度较高，但系统开销也就越大
- **多粒度封锁(Multiple Granularity Locking)**

在一个系统中同时支持多种封锁粒度供不同的事务选择

- 多粒度树
 - 以树形结构来表示多级封锁粒度
 - 根结点是整个数据库，表示最大的数据粒度
 - 叶结点表示最小的数据粒度
- 允许多粒度树中的每个结点被独立地加锁
对一个结点加锁意味着这个结点的所有后裔结点也被加以同样类型的锁
在多粒度封锁中一个数据对象可能以两种方式封锁：显式封锁和隐式封锁
显式封锁：直接加到数据对象上的封锁
隐式封锁：该数据对象没有独立加锁，是由于其上级结点加锁而使该数据对象加上了锁
显式封锁和隐式封锁的效果是一样的。
系统检查封锁冲突时，要检查显式封锁，还要检查隐式封锁

• 选择封锁粒度

同时考虑封锁开销和并发度两个因素，适当选择封锁粒度

- 需要处理多个关系的大量元组的用户事务：以数据库为封锁单位
- 需要处理大量元组的用户事务：以关系为封锁单元
- 只处理少量元组的用户事务：以元组为封锁单位

第十章 关系数据理论

数据依赖：定义属性值间的相互关连（主要体现于值的相等与否），这就是数据依赖，它是数据库模式设计的关键

数据依赖的类型：函数依赖 (Functional Dependency, 简记为 FD)；多值依赖 (Multivalued Dependency, 简记为 MVD)

函数依赖：设 $R(U)$ 是一个属性集 U 上的关系模式， X 和 Y 是 U 的子集。若对于 $R(U)$ 的任意一个可能的关系 r ， r 中不可能存在两个元组在 X 上的属性值相等，而在 Y 上的属性值不等，则称“ X 函数确定 Y ”或“ Y 函数依赖于 X ”，记作 $X \rightarrow Y$ 。

平凡函数依赖与非平凡函数依赖：在关系模式 $R(U)$ 中，对于 U 的子集 X 和 Y ，如果 $X \rightarrow Y$ ，但 $Y \not\subset X$ ，则称 $X \rightarrow Y$ 是非平凡的函数依赖，若 $X \rightarrow Y$ ，但 $Y \subset X$ ，则称 $X \rightarrow Y$ 是平凡的函数依赖。若 $X \rightarrow Y$ ，则 X 称为这个函数依赖的决定属性组，也称为决定因素 (Determinant)。若 $X \rightarrow Y$ ， $Y \rightarrow X$ ，则记作 $X \leftrightarrow Y$ 。若 Y 不函数依赖于 X ，则记作 $X \nrightarrow Y$ 。

完全函数依赖与部分函数依赖：在 $R(U)$ 中，如果 $X \rightarrow Y$ ，并且对于 X 的任何一个真子集 X' ，都有 $X' \nrightarrow Y$ ，则称 Y 对 X 完全函数依赖，记作： $X \twoheadrightarrow Y$ 。若 $X \rightarrow Y$ ，但 Y 不完全函数依赖于 X ，则称 Y 对 X 部分函数依赖，记作 $X \twoheadrightarrow Y$ 。

传递函数依赖：在 $R(U)$ 中，如果 $X \rightarrow Y$ ，($Y \not\subset X$)， $Y \rightarrow X$ ， $Y \rightarrow Z$ ，则称 Z 对 X 传递函数依赖。记为： $X \twoheadrightarrow Z$

码：设 K 为 $R<U, F>$ 中的属性或属性组合。若 $K \rightarrow U$ ，则 K 称为 R 的**候选码** (Candidate Key)。若候选码多于一个，则选定其中的一个做为主**码** (Primary Key)。包含在任何一个候选码中的属性，称为**主属性** (Prime attribute) 不包含在任何码中的属性称为**非主属性** (Nonprime attribute) 或**非码属性** (Non-key attribute) 全码：整个属性组是码，称为**全码** (All-key)。关系模式 R 中属性或属性组 X 并非 R 的码，但 X 是另一个关系模式的码，则称 X 是 R 的外部码 (Foreign key) 也称**外码**。

范式：

1NF 的定义：如果一个关系模式 R 的所有属性都是不可分的基本数据项，则 $R \in 1NF$ ，

2NF 的定义：若 $R \in 1NF$ ，且每一个非主属性完全函数依赖于码，则 $R \in 2NF$ 。

3NF 的定义：关系模式 $R<U, F>$ 中若不存在这样的码 X 、属性组 Y 及非主属性 Z ($Z \not\subset Y$)，使得 $X \rightarrow Y$ ， $Y \rightarrow Z$ 成立， $Y \not\rightarrow X$ ，则称 $R<U, F> \in 3NF$ 。(每一个非主属性既不部分依赖于码也不传递依赖于码)

BC 范式：关系模式 $R<U, F> \in 1NF$ ，若 $X \rightarrow Y$ 且 $Y \not\subset X$ 时 X 必含有码，则 $R<U, F> \in BCNF$ 。(每一个决定属性因素都包含码)，

4NF 的定义：关系模式 $R<U, F> \in 1NF$ ，如果对于 R 的每个非平凡多值依赖 $X \twoheadrightarrow Y$ ($Y \not\subset X$)， X 都含有码，则 $R \in 4NF$ 。

多值依赖：设 $R(U)$ 是一个属性集 U 上的一个关系模式， X 、 Y 和 Z 是 U 的子集，并且 $Z = U - X - Y$ 。关系模式 $R(U)$ 中多值依赖 $X \twoheadrightarrow Y$ 成立，当且仅当对 $R(U)$ 的任一关系 r ，给定的一对 (x, z) 值，有一组 Y 的值，这组值仅仅决定于 x 值而与 z 值无关。

关系模式规范化的基本步骤：

	1NF
	↓ 消除非主属性对码的部分函数依赖
消除决定属性	2NF
集非码的非平	↓ 消除非主属性对码的传递函数依赖
凡函数依赖：	3NF
	↓ 消除主属性对码的部分和传递函数依赖
	BCNF
	↓ 消除非平凡且非函数依赖的多值依赖
	4NF

第十一章 数据库设计

数据库设计的特点：数据库建设的基本规律，三分技术，七分管理，十二分基础数据；结构 (数据) 设计和行为 (处理) 设计相结合。

数据库设计的基本步骤：需求分析 – 概念结构设计 – 逻辑结构设计 – 物理结构设计 –

数据库实施 – 数据库运行和维护

需求分析的方法：调查需求，达成共识，分析表达需求

调查用户需求的具体步骤：(1) 调查组织机构情况；(2) 调查各部门的业务活动情况；(3) 在熟悉业务活动的基础上，协助用户明确对新系统的各种要求；(4) 确定新系统的边界

常用调查方法：(1)跟班作业，(2)开调查会，(3)请专人介绍，(4)询问，(5)设计调查表请用户填写，(6)查阅记录

概念结构设计的方法与步骤：自顶向下，自底向上，逐步扩张，混合策略

E-R 图向关系模型的转换：

转换内容：将 E-R 图转换为关系模型：将实体、实体的属性和实体之间的联系转换为关系模式。实体型间的联系：(1)一个 1:1 联系可以转换为一个独立的关系模式，也可以与任意一端对应的关系模式合并。(2)一个 1:n 联系可以转换为一个独立的关系模式，也可以与 n 端对应的关系模式合并。(3) 一个 m:n 联系转换为一个关系模式。(4)三个或三个以上实体间的一个多元联系转换为一个关系模式。(5)具有相同码的关系模式可合并

数据库的运行与维护：数据库的转储和恢复；数据库的安全性、完整性控制；数据库性能的监督、分析和改进；数据库的重组织和重构

数据库视图集成的两种方式：多个分 E-R 图一次集成；逐步集成

第十章 数据库恢复技术

第十一章 并发控制

并发控制机制的任务：对并发操作进行正确调度，保证事务的隔离性，保证数据库的一致性

并发操作带来的数据不一致性：丢失修改 (Lost Update)；不可重复读 (Non-repeatable Read)；读“脏”数据 (Dirty Read) 数据不一致性：由于并发操作破坏了事务的隔离性

并发控制的主要技术：有封锁(Locking)，时间戳(Timestamp)，乐观控制法

封锁就是事务 T 在对某个数据对象（例如表、记录等）操作之前，先向系统发出请求，对其加锁

基本封锁类型：**排它锁** (Exclusive Locks，简记为 X 锁) (写锁) 若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A，其它任何事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁，保证其他事务在 T 释放 A 上的锁之前不能再读取和修改 A；**共享锁** (Share Locks，简记为 S 锁) 共享锁又称为读锁。若事务 T 对数据对象 A 加上 S 锁，则其它事务只能再对 A 加 S 锁，而不能加 X 锁，直到 T 释放 A 上的 S 锁，保证其他事务可以读 A，但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改。

可串行化调度：多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同

两段锁协议：两段封锁协议(Two-Phase Locking，简称 2PL)是最常用的一种封锁协议，理论上证明使用两段封锁协议产生的是可串行化调度。两段锁协议：指所有事务必须分两个阶段对数据项加锁和解锁

三级封锁协议的内容：

- ◆ 一级封锁协议：事务 T 在修改数据 R 之前必须先对其加 X 锁，直到事务结束才释放。一级封锁协议可以防止丢失修改，并保障事务是可恢复的。
- ◆ 二级封锁协议：一级封锁协议加上事务 T 在读取数据 R 之前必须先对其加 S 锁，数

据读完后可释放 S 锁。本协议防止了丢失修改，而且进一步防止了读肮脏数据。

- ◆ 三级封锁协议：一级封锁协议加上事务 T 在读取数据 R 之前必须先对其加 S 锁，直到事务读完后才释放。本协议可以解决丢失修改和不读肮脏数据、还防止了不可重复读数。